NeoAI
Yapay Zekâ Destekli Ultrason İnceleme

TOBB ETÜ
University of Economics & Technology

BİL 495/496 Bitirme Projesi
Final Raporu

Final Stack

Ali Belikırık
Beyza Değirmenci
İnan Özşahin
Özge Doğan

Summary
This final report presents ***NeoAI***, a web-based clinical decision support system designed to support diagnosis of diseases using multi-region ***lung ultrasound***. The system lets authenticated clinicians move from ***patient and examination*** lookup through ***frame and region selection***, ***configurable preprocessing***, and ***AI-assisted analysis***, ending in structured reporting and review.
The system uses a full-stack architecture, integrating a **React** application, a ***Spring Boot API***, ***PostgreSQL*** for persistence and simulating ***Hospital Information System***, ***Google Cloud Storage*** for media, and a ***Python FastAPI*** service that will run computer-vision models for tasks such as ***B-line detection***, ***RDS scoring***, and disease classification depending on selected modules.

Security is addressed through **JWT-based authentication** and r**ole-based access control** (e.g. doctor, admin roles). To support scalability and responsiveness, the platform implements an **asynchronous** analysis pipeline: analysis requests are submitted to the AI service, which processes ultrasound frames via URL references and returns results through callback endpoints. This approach ensures that long-running inference tasks do not block user interaction. The database schema is designed to maintain analysis jobs, module executions, and associated metadata, enabling auditability, status tracking, and efficient retrieval of results.

The report systematically presents the system architecture, evaluates the impact of the engineering solutions developed, and discusses current regulatory and ethical considerations, along with the tools used, background research, and testing methodology

Key words: ***Lung ultrasound, clinical decision support systems, artificial intelligence in healthcare, computer vision, medical image analysis, B-line detection, respiratory distress syndrome, disease classification, web-based healthcare systems, asynchronous processing***

Summary 2
1. Introduction 4
2. Problem Statement and Project Objectives 5
2.1 Problem Statement 5
2.2 Project Objectives 5
2.3 Scope and Limitations 6
3. System Architecture and Design 6
3.1 Overall Architectural Approach 6
3.2 Server Side (Backend) 7
3.3 Data Model and Persistence 8
3.4 Cloud Storage and Media Access 8
3.5 Frontend 9
3.7 Final Architecture State 13
4. Impact Analysis 14
4.1 Global Aspect 14
4.2 Economic aspect 15
4.3 Environmental Aspect 15
4.4 Social Aspect 15
5. Current Topic 16
5.1 Regulation of medical software and artificial intelligence 16
5.2 Data Protection (KVKK) 16
5.3 Security and Cyber Threats 16
5.4 Algorithmic reliability and bias 16
5.5 Transparency and clinical confidence 16
5.6 Human oversight and professional ethics 17
6. Tools and Technologies 17
6.1 Server Side (Backend) 17
6.2 Client side (frontend) 18
6.3 Infrastructure and deployment 18
6.4 Geliştirme araçları 19
6.5 Technology Choice Rationale 19
7. Background Research 19
7.1 Similar designs and the literature in the field 20
7.2 Component and technology resources 20
7.3 Basic engineering principles 20
8. Test Results 21
8.1 Test environment and prerequisites 21
8.2 Test scope 21
8.3 Test cases and results 21
8.4 Test evaluation 23
9.Conclusions and Recommendations 24
10.Appendices 24
11.Reference 28

1. Introduction

Medical imaging plays a critical role in Neonatal Intensive Care Unit (NICU) practice, where rapid and accurate decision-making is essential for the care of critically ill newborns. In this context, ultrasound is one of the most frequently used imaging modalities due to its non-ionizing nature, portability, and suitability for bedside use in fragile patients. Within NICU workflows, especially in multi-region ultrasound acquisition protocols, clinicians must carefully select clinically relevant frames for each anatomical region, apply consistent preprocessing steps, and optionally leverage AI-assisted analysis tools to support interpretation. These processes not only improve diagnostic efficiency and consistency but also enhance traceability for clinical documentation, archiving, and quality assurance. As a result, there is a clear need to integrate these fragmented and often manually handled tasks into a unified, digital, standardized, and secure workflow tailored to the NICU environment.

From an information technology perspective, a NICU-focused platform must go beyond simple image visualization. It should support secure authentication mechanisms, enforce strict role-based access control appropriate for sensitive neonatal data, and organize patient examinations and imaging data in a structured and consistent manner. Additionally, all analysis activities should be recorded in a traceable way within a persistent database to ensure clinical accountability and reproducibility. Rather than transferring large video and image datasets through the application server, a more scalable and secure approach is to store media in object storage and distribute access via time-limited, signed URLs. On the client side, the inherently time-critical and multi-step nature of NICU clinical workflows requires a streamlined and intuitive user experience, which is best achieved through a single-page application architecture.

The NeoAI platform developed within the scope of this project is designed specifically to address the requirements of NICU-based ultrasound workflows. The project was carried out by the FinalStack team as part of the BIL495 and BIL496 graduation project, with both backend and frontend components implemented as an integrated engineering system. The aim of this report is to present the final system architecture and design in detail; to evaluate the impact of the proposed solution in global, economic, environmental, and social contexts; to summarize current challenges in the field; to document the tools and technologies used; to outline the supporting research conducted through academic and online sources; and to present the testing strategy and evaluation results in accordance with the defined test plan. In this way, the report aims both to satisfy the academic requirements of the course and to clearly communicate the technical and clinical relevance of the system.

In the following sections, the problem definition and project objectives are presented, followed by the system architecture, impact analysis, current challenges, tools and technologies, research summary, and test results.

2. Problem Statement and Project Objectives

2.1 Problem Statement

In neonatal care, lung ultrasound is frequently used at the bedside to support rapid diagnosis and monitoring of newborn patients. However, interpreting these scans can be challenging, particularly for less experienced clinicians. Differences in experience may lead to variation in how images are selected and interpreted, which can affect consistency in clinical decisions. In addition, clinicians working under pressure or in emergency settings may benefit from a second opinion to increase confidence in their assessments.

Another challenge is that ultrasound data is often reviewed in a fragmented way, with images, notes, and decisions stored separately. This makes it harder to revisit cases, compare findings, or ensure consistent follow-up. As a result, valuable clinical information may not be fully utilized, and decision-making can become slower or less standardized.

To address these challenges, there is a need for a system that supports clinicians by helping them organize and review ultrasound data more effectively, while also providing AI-assisted guidance to speed up and support decision-making when needed.

2.2 Project Objectives

The primary objective of this project is to support a multi-step, ultrasound-based clinical workflow through a web-based application. In line with this goal, the following objectives have been defined:

Secure access and user management

Ensuring secure user authentication; managing sessions using a stateless, token-based approach; and enabling administrators to create and update users, thereby maintaining full control over system user lifecycle management.

Examination and media access

Listing patient-related examination videos; securely delivering media objects at both examination and region level through cloud storage using time-limited signed URLs; and generating signed upload URLs when necessary.

Structured clinical workflow

Providing a clear, step-by-step workflow on the client side, including patient selection, representative frame selection from regional videos, configuration of preprocessing steps, selection of AI modules, and a final review and reporting stage.

Recording of analysis sessions

Storing clinician inputs and analysis identifiers on the server side; enabling tracking of analysis status; and keeping results in a structured and extensible format (e.g., JSON) to support future enhancements.

Extensible architecture

Designing the system in a modular way so that future additions—such as real AI inference services, hospital information systems, or additional imaging modalities—can be integrated without major changes to the existing structure.

2.3 Scope and Limitations

This project is designed as a focused web-based application that supports a specific neonatal ultrasound workflow, rather than functioning as a full Hospital Information System (HIS) or Picture Archiving and Communication System (PACS). Its primary purpose is to provide structured support for clinicians during the review and analysis of neonatal lung ultrasound data.

Most features of the project operate using sample data until full integration with external systems is completed. The development team did not have access to the hospital's information systems during development. In a production environment, these components are intended to be connected to real hospital data sources and clinical systems.

The project also represents the initial step toward a larger AI-driven diagnostic support platform for Neonatal Intensive Care Units (NICU). While the current scope focuses on ultrasound-based analysis, the long-term vision includes integrating additional clinical data sources such as ECG signals, MRI scans, and pulse oximetry measurements to enable more comprehensive decision support.

Integration of advanced AI modules is expected to be incremental due to technical, infrastructural, and regulatory constraints. As a result, certain AI-driven functionalities may be partially implemented or simulated within the current version of the system. The final sections of this report clearly specify which components are fully operational and which are planned for future expansion.

3. System Architecture and Design

3.1 Overall Architectural Approach

The system is designed using a multi-tier, service-oriented architecture that separates the user interface, application logic, data storage, and AI processing components.

Web Application Layer

The web application layer consists of a React single-page application (SPA) that communicates with a Spring Boot REST API over HTTPS. Authentication is handled using JWT (JSON Web Token), which secures protected endpoints and enables stateless session management.

Backend Application Layer

The backend application layer is responsible for orchestrating the core clinical workflow. This includes managing patient and examination context, user preferences, and the full lifecycle of analysis operations. It also integrates external services through HTTP-based communication.

Data Persistence Layer

The data persistence layer is implemented using PostgreSQL, which stores both relational and JSON-based data structures. This includes users, patients, analyses, preprocessing configurations, module execution records, and audit logs.

Media Storage Layer

The media storage layer uses Google Cloud Storage (GCS) to store ultrasound examination videos. The backend generates time-limited signed URLs, allowing secure access to media files without exposing long-term credentials to clients or external services.

AI inference layer

The AI inference layer is implemented as an external FastAPI service (ai_module). This service executes asynchronous analysis tasks such as B-line detection (based on a YOLO model) and other scoring modules. It communicates with the backend through a defined HTTP contract and uses callback mechanisms to return results. Currently the ai-module is not fully developed.

This separation ensures that clinical logic, user interaction, and AI processing remain decoupled, allowing the AI service to be independently updated or replaced without affecting the core system.

3.2 Server Side (Backend)

The backend is implemented in Java 21 using Spring Boot and exposes versioned REST APIs under the /api/v1/... namespace.

Key responsibilities include:

Security:
The system uses stateless JWT-based authentication. Most endpoints are protected, while the AI callback endpoint is explicitly excluded from authentication to allow the external AI service to submit results. CORS configuration supports standard local development environments for the React frontend.

AI Orchestration:
When an analysis request is initiated, the backend creates a main analysis entity (UsAiAnalysis) along with separate module execution records (UsAnalysisModuleRun) for each lung region (R1–R6). For each region, the backend retrieves a signed video URL from GCS and sends a request to the AI service using Spring WebClient. Each request includes the frame index, selected modules, and a callback URL. The returned job_id values are stored for later correlation.

Callback Handling:
The endpoint /api/v1/ai-analysis/callback receives results from the AI service. It updates the corresponding module execution record and checks whether all regions have completed. Once all tasks are finished, the overall analysis status is updated.

Supporting Services:
Additional functionality includes patient and examination management, preprocessing configuration handling, and cloud storage utilities.

Configuration values such as database connection, GCS bucket information, AI service URL, and callback URL are managed through environment-based properties.

3.3 Data Model and Persistence

The system uses PostgreSQL (neoai_db) as its primary database. The schema is initialized using Docker scripts and managed through JPA/Hibernate.

The data model is organized into the following logical domains:

User and Access Management:
Includes user accounts, roles, and optional access restrictions based on data types.

Clinical Structure:
Represents patients and their examinations, which serve as the foundation for linking imaging data and analysis workflows.

AI Configuration:
Includes a catalog of AI modules (e.g., B-line detection, RDS scoring) and preprocessing operations. Default configurations are stored in JSON format, with support for user-specific overrides.

Analysis Execution:
Stores analysis sessions (us_ai_analyses), per-region processing tasks (us_analysis_module_runs), frame selections, preprocessing parameters, and AI results. This structure ensures traceability of each analysis step.

Audit Logging:
Records user actions for traceability and system monitoring.

The AI service maintains its own lightweight database (analysis_jobs) for tracking asynchronous inference tasks. This table is linked to the backend through shared job_id values, enabling result synchronization without duplicating the full clinical data model.

3.4 Cloud Storage and Media Access

Google Cloud Storage (GCS) is used as the primary storage solution for ultrasound examination media.

Structured Object Organization:
Media files are stored using a hierarchical naming convention such as:
ai/PT_{patientId}/{examinationName}/{R1..R6}.mp4
This structure supports efficient retrieval based on patient and examination context.

Signed URL Access:
The backend generates time-limited signed URLs for secure access. GET URLs allow clients and the AI service to stream videos, while PUT URLs can be used for controlled uploads when required. This avoids exposing permanent credentials.

Integration Layer:
REST endpoints under /api/v1/assets provide access to stored media and support listing or filtering based on bucket prefixes.

3.5 Frontend

The frontend is implemented as a React single-page application designed around a step-based clinical workflow.

Main features include:

- User authentication and session handling
- Patient and examination selection
- Region-based video review and frame selection (R1–R6)
- Preprocessing configuration interface
- AI module selection (e.g. B-line detection, RDS scoring)
- Results visualization and reporting
- Administrative and audit views (where enabled)

Technical implementation details:

- Axios is used for HTTP communication with the backend
- API base URL is configured through environment variables
- JWT tokens are stored and attached via request interceptors
- Protected routes ensure role-based access control
- Video utilities support frame extraction and previewing for clinical review

3.6 End-to-End Data Flow

1. ***Application Initialization***

   The user accesses the system through a web browser. The React application (Vite-based) is initialized via main.jsx, which sets up routing and the UI framework (MUI). The application defines multiple workflow-based routes, including:

   /login, /query, /selection/..., /preprocessing/..., /ai-module/..., /results/..., and /report/....

   These routes represent the step-by-step clinical workflow implemented in App.jsx.

2. ***Authentication and API Communication***

   The user logs in through the backend authentication endpoint (POST /api/v1/auth/login). Upon successful login, a JWT token is stored in localStorage.

   All subsequent API requests are handled through a shared Axios client (httpClient.js), which automatically attaches the header:

   Authorization: Bearer <token>

Requests are sent to the backend base URL defined by VITE_API_BASE_URL (default: http://localhost:9090).

3. ***Patient and Examination Retrieval***

To retrieve patient data, the frontend calls:

GET /api/v1/examinations/{patientId}

The ExaminationVideoListingController processes this request by querying the service layer, which interacts with Google Cloud Storage (GCS) to retrieve a paginated list of examinations and their metadata.

4. ***Loading Examination Media***

When a specific examination is selected, the frontend requests:

GET /api/v1/examinations/{patientId}/{timeAndExamName}

This returns a list of ExaminationVideoDTO objects containing metadata and time-limited signed URLs for each anatomical region. The frontend maps this data into the selection and preprocessing workflow.

5. ***Clinical Workspace (Client-Side Processing)***

In the selection, preprocessing, and AI module screens, the clinician:

- Selects representative frames for each region (R1–R6)
- Configures optional preprocessing steps
- Selects AI analysis modules (e.g. B-line detection, RDS scoring)

Most of this state is managed on the client side using React hooks. In some cases, OpenCV.js is used for local image manipulation and previewing before analysis submission.

6. ***Starting an Analysis Request***

When the workflow is completed, the frontend sends a request:

POST /api/v1/ai-analysis

This request includes:

- Examination ID and patient ID
- Selected frame indices per region
- Selected AI modules
- Preprocessing configuration

7. ***Backend Analysis Orchestration***

The backend service (AiAnalysisServiceImpl) performs the following steps:

- Loads and validates the examination and patient data
- Creates a UsAiAnalysis entity with initial status
- Stores per-region analysis configuration
- Creates module execution records (UsAnalysisModuleRun)
- Initiates AI processing via AiModuleIntegrationService

8. ***AI Service Invocation***

The integration service (AiModuleIntegrationServiceImpl) handles communication with the AI module:

- Marks the analysis as *PROCESSING*
- Retrieves signed video URLs from GCS for each region
- Constructs a request containing:
    - video URL
    - selected frame index
    - selected AI modules
    - callback URL
- Sends a POST /analyze request to the FastAPI service
- Stores the returned job_id for tracking

The system may poll GET /jobs/{jobId} to monitor job completion.

9. ***AI Service Execution (FastAPI)***

The AI module performs asynchronous processing:

- Creates a job entry in its analysis_jobs table
- Executes inference (e.g. B-line detection, RDS scoring logic)
- Produces structured JSON results
- Updates job status in the database

The backend retrieves results via polling and integrates them into the analysis record.

10. ***Result Aggregation and Reporting***

Once all regions are processed, the backend:

- Merges results into a unified resultData structure
- Updates analysis and module run entities
- Marks the analysis as *COMPLETED* or *FAILED*

The frontend retrieves results via:

GET /api/v1/ai-analysis/{uuid}

Physician notes and interpretations can be managed using:

- GET /api/v1/ai-analysis/{uuid}/doctor-suggestion

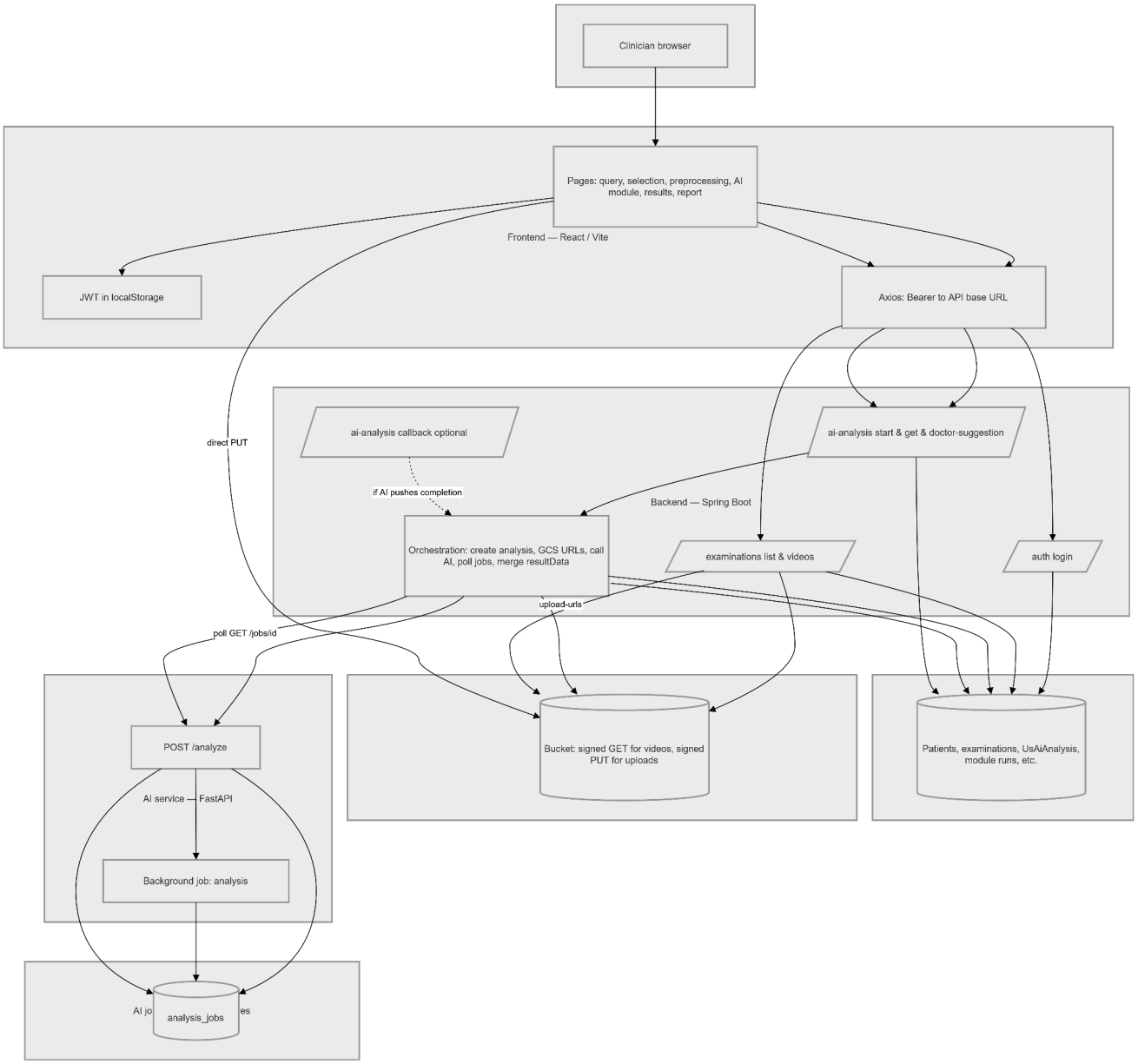
- PUT /api/v1/ai-analysis/{uuid}/doctor-suggestion

11. Optional Media Upload Workflow

For cases requiring uploads, the backend provides signed upload URLs via:

POST /api/v1/examinations/{patientId}/{examName}/upload-urls

These URLs allow direct uploads to GCS under structured paths such as:ai/PT_{patientId}/{examination}/{region}



Data flow diagram

3.7 Final Architecture State

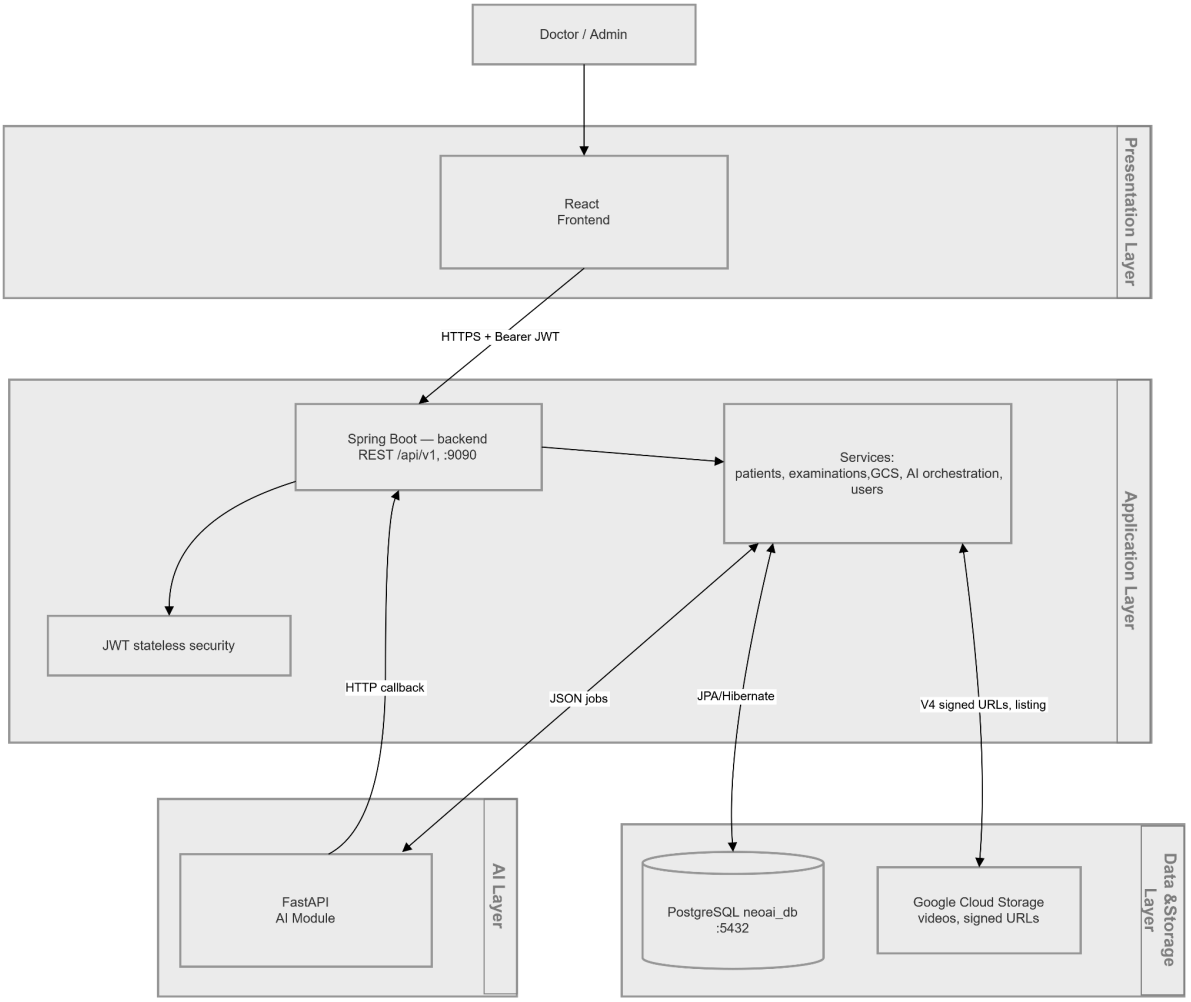
The system is deployable using Docker Compose for PostgreSQL and the AI service, while the Spring Boot backend can run either locally or separately. In development setups, services communicate using host-based routing (e.g. host.docker.internal) to enable callback delivery.

The current architecture reflects a modular separation of responsibilities:

- Backend and frontend are fully developed in-house
- AI module is an external microservice integrated via HTTP

- PostgreSQL serves both operational and analytical data storage
- GCS provides scalable media storage

The system is currently optimized for a six-region neonatal lung ultrasound workflow. Future extensions are expected to reuse the same modular pattern for additional AI modules and clinical data types.



Final system architecture

4. Impact Analysis

4.1 Global Aspect

Digital transformation in healthcare and AI-powered imaging are at the center of global discussions regarding access, quality, and data sovereignty. Integrating protocols such as multi-regional ultrasound into a standardized workflow facilitates the consistent implementation of the same steps across different centers, thereby laying the groundwork for multi-center clinical trials and training programs. On the other hand, cloud-based storage and international service providers raise global compliance requirements, such as where data is stored and compliance with regulations like the EU GDPR or

similar frameworks. NeoAI's architecture, with its layered structure that separates data from the application server, can be adapted to support local hosting or regional cloud options in the future; this enhances the potential for compliance with diverse regulatory environments on a global scale.
The NeoAI platform that has been developed is not merely a technical software product; it is an engineering solution that could have indirect impacts on healthcare, the economy, the environment, and society. In this section, these impacts are examined separately under the headings of global, economic, environmental, and social impacts.

4.2 Economic aspect

On the cost side, software development, cloud storage, bandwidth, and potential AI inference infrastructure (e.g., GPU usage) for hospitals and clinics generate both direct and indirect expenses. On the benefit side, reducing the time spent on repetitive manual reviews, the administrative efficiency gained from protocol-compliant record-keeping, and the reduced risk of unnecessary repeat imaging or incorrect archiving can create economic benefits. Particularly in units with high patient volume, such as intensive care and emergency departments, reducing the image review workload per clinician can contribute to more efficient use of personnel costs in the long term. To quantitatively measure these effects, however, future field studies or pilot program data based on usage durations and error rates in a production environment will be required.

4.3 Environmental Aspect

Digital health systems do not directly produce emissions; however, data centers, cloud storage, and video processing workloads create an indirect environmental impact through energy consumption. On the NeoAI side, the processing and repeated playback of long ultrasound videos utilize client and server resources; when model inference is added, the energy intensity of GPU-based services increases. Engineering options to reduce environmental impact include efficient video encoding, avoiding unnecessary high resolution, cold storage tiers, and prioritizing cloud regions that prioritize the use of locally available renewable energy. While not all of these measures may have been implemented at this stage of the project, the architecture is designed to reduce the need for unnecessary file copying by storing media in object storage rather than on the application server.

4.4 Social Aspect

Patient privacy and trust are critical to the social acceptance of any system that processes health data. NeoAI's measures—such as JWT and role-based access control, password hashing, and time-stamped URLs—are designed to mitigate the risk of unauthorized access; however, legal processes such as explicit consent, purpose limitation, and retention periods under the KVKK must also be implemented in production. AI outputs should be presented socially at the "recommendation" level; the responsibility for the final clinical decision must remain with the physician; otherwise, issues regarding patient trust and professional liability may arise. From an equity perspective, interfaces requiring high-speed internet and modern browsers may deepen the digital divide in regions with weak infrastructure; therefore, modes capable of operating on low bandwidth or low-resolution previews represent important areas for future improvements in social accessibility. Finally, a standardized workflow and recorded analysis sessions can generate societal benefits for education and quality assurance, as they facilitate clinicians' adherence to protocols and case discussions.

5. Current Topic

AI-powered medical imaging and clinical workflow software are at the center of current debates not only in terms of technical performance metrics but also regarding regulation, patient rights, cybersecurity, and professional ethics. This section summarizes key topics that emerge in the context of a platform like NeoAI.

5.1 Regulation of medical software and artificial intelligence

Regulation of Medical Software and Artificial Intelligence
Under the Medical Device Regulation (MDR) in the European Union, software may be subject to different obligations depending on its risk class. The Artificial Intelligence Act (AI Act), on the other hand, introduces requirements such as data governance, technical documentation, human oversight, and transparency for high-risk artificial intelligence systems. In the United States, the FDA has developed specific evaluation approaches for AI-enabled SaMD (Software as a Medical Device) products. In Turkey, medical device software and market processes are regulated under the TİTCK; the product's classification and the need for clinical evaluation depend on the claimed medical purpose. Which of NeoAI's modules will be classified as "diagnostic support" or "clinical decision support" will directly influence the future regulatory roadmap.

5.2 Data Protection (KVKK)

In Turkey, pursuant to Law No. 6698 on the Protection of Personal Data (KVKK), health data falls under the category of special-category personal data. Legal grounds, explicit consent, data minimization, purpose limitation, and secure storage obligations must be taken into account. For videos and analysis logs stored in the cloud, a processing inventory, retention periods, and deletion policies must be defined. Technical measures such as JWT-based authentication and signed URLs are not sufficient on their own; corporate processes and legal compliance are also mandatory.

5.3 Security and Cyber Threats

Healthcare organizations are high-value targets for ransomware and data breaches. Web applications face threats such as XSS, CSRF (the risk profile varies when using JWT), API abuse, and unauthorized token usage. While storing tokens in the browser offers convenience, implementing strict content security policies to mitigate XSS risks and evaluating alternatives such as HttpOnly cookies in production may be necessary. Limiting the duration of signed URLs is intended to reduce the risk associated with shareable links.

5.4 Algorithmic reliability and bias

Artificial intelligence models may perform better or worse in certain populations depending on the representativeness of the training dataset. Model performance may decline over time due to distribution drift. In clinical settings, calibration, periodic revalidation, and error analysis processes are now essential requirements. Since the NeoAI architecture is designed to record analysis sessions and result structures, it can provide a technical foundation for such traceability and audit processes in the future.

5.5 Transparency and clinical confidence

Clinicians find it difficult to trust AI outputs without understanding why they were generated. In imaging modalities such as ultrasound, forms of explainability—such as region-level scores, heat maps, or brief explanatory texts—can help build trust. On the other hand, overly complex explanations can increase the clinical workload; therefore, it is important to present information in a way that aligns with the user experience and clinical workflow.

5.6 Human oversight and professional ethics

When artificial intelligence is used as an auxiliary tool in medical imaging, the responsibility for the final decision remains with the physician. It is ethically important to clearly explain the role of artificial intelligence in communication with the patient. In emergency situations, the system's uninterrupted operation and backup procedures are critical to patient safety.

6. Tools and Technologies

The solution is a three-tier style setup: a browser SPA talks to a Java API, which persists data in PostgreSQL, calls an asynchronous AI microservice (Python/FastAPI), and integrates Google Cloud Storage for media. Local development is orchestrated with Docker Compose (database + AI container).

Main Components

Layer	**Role**	**Primary Stack**
Presentation Layer	Clinical workflow, auth UX, reporting-oriented UI	React 18, Vite 5, MUI 7
Application Layer	REST, security, orchestration, persistence	Spring Boot 3.4, Java 21
AI Layer	Analysis jobs, callbacks	Python 3.11, FastAPI, Uvicorn, asyncpg
Data Layer	Relational store + JSON for flexible payloads	PostgreSQL 16
Storage Layer	Signed URLs, listing	Google Cloud Storage (Java client)

6.1 Server Side (Backend)

- ***Primary API***

  Language: Java 21. It was chosen because it is part of the Long-Term Support (LTS) line and offers modern language features.

  Framework: Spring Boot 3.4.x (e.g., 3.4.3). It provides embedded server support, dependency management, and production-ready configuration with minimal setup.

  Spring modules:

- spring-boot-starter-web: REST API and HTTP layer.
- spring-boot-starter-data-jpa: Entity–relationship mapping and repository abstraction.
- spring-boot-starter-security: Authentication and authorization.
- spring-boot-starter-validation: Validation rules for request bodies.

Database: PostgreSQL (e.g., version 16, via Docker image). It supports relational data integrity and flexible schema needs through JSON/JSONB fields.

Authentication: JWT generation and validation using JJWT (e.g., 0.11.5); used for stateless API sessions.

Cloud storage: Google Cloud Storage Java client (google-cloud-storage, e.g., 2.35.0) for object listing and generating signed GET/PUT URLs.

Utility libraries: Lombok (reduces boilerplate code such as getters and setters).

Build & dependency management: Maven (pom.xml).

- ***AI API***

6.2 Client side (frontend)

Kütüphane: React 18.3.x — bileşen tabanlı kullanıcı arayüzü.
Derleme ve geliştirme sunucusu: Vite 5.4.x — hızlı geliştirme ve üretim derlemesi.
Yönlendirme: react-router-dom 6.28.x — çok sayfalı iş akışı rotaları.
Arayüz bileşenleri: Material UI (MUI) 7.3.x ve Emotion — tutarlı tema ve bileşen seti.
HTTP istemcisi: Axios 1.14.x — API çağrıları ve istek öncesi JWT ekleme.
PDF üretimi: @react-pdf/renderer 4.4.x — rapor çıktısı için istemci tarafı belge oluşturma.

6.3 Infrastructure and deployment

Local / developer environment
Docker Compose (backend/docker-compose.yml):

PostgreSQL 16 : named volume postgres_data, port 5432, DB neoai_db, init scripts from backend/docker/postgres/init/
AI module : build context ../ai_module, port 8001 on host mapped to 8000 in container; depends on healthy Postgres; host.docker.internal used so the container can call the Spring app on the host; DATABASE_URL points at the compose Postgres service.
Host processes: Spring Boot (Maven/IDE) and npm run dev for the frontend are expected to run on the host while Compose provides DB + AI

Cloud / external integration
Google Cloud Storage : project and bucket configured in application.properties; credentials loaded from classpath:gcp-key.json

Ports

Service	*Port*
React(Vite dev)	5173
Spring Boot	9090
PostgreSQL	5432
AI module (Docker)	8001 → 8000

6.4 Geliştirme araçları

IDE: IntelliJ IDEA veya benzeri Java IDE; React için VS Code veya benzeri düzenleyici.
Sürüm kontrolü: Git — kaynak kodunun izlenmesi ve ekip çalışması.

6.5 Technology Choice Rationale

Spring Boot (Java) for the main API : Strong ecosystem for REST, security, JPA, and integration with enterprise systems; clear structure for a clinical workflow backend; WebClient fits async HTTP to the AI service without blocking servlet threads.

PostgreSQL: Robust relational model for users, examinations, and analyses, plus JSON/JSONB for flexible AI payloads and settings; matches both JPA and asyncpg usage.

JWT + Spring Security: Stateless API suitable for SPAs; role and allowed data types encoded in tokens supports authorization without server-side sessions.

Separate FastAPI service for AI: Python is standard for ML/CV pipelines; FastAPI gives fast iteration, OpenAPI-friendly APIs, and async I/O; isolation lets you scale or replace the AI stack independently of the Java monolith.

React + Vite + MUI: React is widely adopted for complex UIs; Vite gives fast dev feedback and lean production builds; MUI speeds up accessible, consistent medical-style interfaces and reduces custom CSS burden.

Docker Compose for dependencies: One command brings up Postgres + AI with consistent versions; developers run only the JVM and Node tooling on the host.

Google Cloud Storage: Fits large video assets and signed upload/download URLs, offloading binary traffic from the application server.

7. Background Research

Throughout the project, the design and implementation of the NeoAI platform were not limited to writing code; instead, libraries and online resources were reviewed in accordance with similar systems, the official documentation of the components used, and fundamental engineering principles.

This section summarizes the scope of the research under three headings and lists examples of resource types.

7.1 Similar designs and the literature in the field

In bedside ultrasound and lung ultrasound applications, multi-site protocols, clinical scoring, and repeatable imaging steps are widely discussed. Reviews and guideline-type publications related to point-of-care ultrasound, B-line assessment, and aeration were examined through academic databases and clinical journals. These sources do not directly define the software architecture; however, they provide context regarding the necessary steps in the workflow (region selection, representative frames, clinician notes).

In a hospital setting, web viewers integrated with PACS and HIS operate via DICOM or proprietary archives. Since NeoAI is positioned as a lightweight web workflow rather than a full-fledged PACS, the heavy integration examples in the literature were adopted as a "reference architecture"; the project, however, was limited to a simpler solution focused on REST APIs and object storage.

On the AI-assisted medical imaging side, classification, segmentation, and decision support studies provide insights into dataset diversity, labeling costs, and the need for clinical validation. These studies lay the groundwork for expectation management and risk considerations regarding the future integration of an external model service into the system.

7.2 Component and technology resources

The following resources are official and reliable secondary sources for the technologies used directly in the project:

Spring Boot / Spring Security / Spring Data JPA: The official reference documentation for the Spring ecosystem; serves as a reference for REST endpoints, the security chain, JPA entity mapping, and transactional boundaries.
PostgreSQL: Version documentation; provides resources for the relational model, indexing, and the use of JSON/JSONB.
Google Cloud Storage: Google Cloud documentation was used for signed URLs (V4), bucket listing, and object metadata.
React and Vite: Official guides for component lifecycle, routing, and production builds.
Material UI: Documentation for accessibility and consistent component usage with the theme.
JWT (RFC 7519): Standard text regarding token structure and transport format; implementation details are supported by the documentation of the library used.
These resources form the foundation not only for understanding "how it works" but also for secure configuration (e.g., CORS, signed URL durations).

7.3 Basic engineering principles

The following principles have been supported by the literature and engineering practice within the scope of this research:

Layered architecture: Separation of presentation, application, and data access; reduces the cost of change.

Separation of concerns: Clear separation of controller, service, and repository responsibilities.
Stateless server: A JWT-based session model for ease of scaling and deployment; in exchange, client-side token security requirements.
Data minimization and purpose limitation: A design goal aligned with the KVKK perspective.
Secure defaults: Hashing of passwords, time-limited signed URLs, and server-side validation of role-based administrative operations.

8. Test Results

8.1 Test environment and prerequisites

Object	Description
Client	A modern Chromium-based browser (e.g., Google Chrome or Microsoft Edge)
Client build	Development server (npm run dev) or production preview (npm run build + preview)
Server	Spring Boot application, default URL: http://localhost:8080
Environment variables	PostgreSQL (with Docker Compose)
Ortam değişkenleri	API base URL for the client (VITE_API_BASE_URL); GCP credentials and bucket configuration for tests requiring GCS
Test data	Sample user accounts (demo accounts or database seed data as specified in the documentation)

8.2 Test scope

The tests focus primarily on functional aspects: authentication, authorization, user and profile operations, inspection and media APIs (if applicable), initiating and querying analytics, client workflow routes, and log viewing. Automated unit tests may be limited or absent in this report; the results are primarily based on manual or semi-automated verification.

8.3 Test cases and results

Test ID	Description	Input(s)	Expected Output	Result
TC-01	Doctor login with valid credentials	Valid username & password	Login successful; main menu displayed	Pass
TC-02	Doctor login with invalid credentials	Wrong password	Error message shown; access denied	Pass

Test ID	Description	Input(s)	Expected Output	Result
TC-03	Admin login with valid credentials	Admin username & password	Login successful; main menu + Admin Panel available	Pass
TC-04	Unauthorized access attempt without login	Direct URL access	Redirect to login page	Pass
TC-05	Query patient data by valid patient ID	Existing patient ID	List of examinations with ID and date shown	Pass
TC-06	Query patient data by non-existent ID	Non-existent patient ID	Appropriate 'no records found' message	Pass
TC-07	Select an examination from the list	Click on examination entry	Videos for all regions (R1-R6) displayed	Pass
TC-08	Select one image per region (R1-R6)	Select 1 image from each region video	6 images selected; approve button enabled	Pass
TC-09	Attempt to proceed without selecting all regions	Select images for only 4 of 6 regions	Warning message; progression blocked	Pass
TC-10	Approve image selection	All 6 regions selected, click approve	Preprocessing panel opened	Pass
TC-11	Load preprocessing settings panel	Approved image selection	Preprocessing operations panel displayed	Pass
TC-12	Apply preprocessing operation (e.g., noise reduction)	Select noise reduction, set intensity=5	Operation listed with parameters	Pass
TC-13	Change preprocessing operation settings	Modify existing operation parameter	Updated value reflected in settings panel	Pass
TC-14	Remove a preprocessing operation	Deselect an applied operation	Operation removed from list	Pass
TC-15	Select AI model for diagnosis	Choose available AI model from list	AI model selected; ready to process	Pass
TC-16	Proceed without selecting an AI model	Click continue with no AI selected	Error prompt; progression blocked	Pass
TC-17	System applies preprocessing and sends to AI	Valid images + operations + AI model	Processing completes; report generated	Pass
TC-18	View generated diagnostic report	Completed processing	Report panel displayed with AI results	Pass
TC-19	Add commentary to the report	Type commentary text in report panel	Commentary saved and visible in report	Pass
TC-20	Export report as PDF	Click export, select PDF	PDF file downloaded successfully	Pass
TC-21	Export report as DOC	Click export, select DOC	DOC file downloaded successfully	Fail

Test ID	Description	Input(s)	Expected Output	Result
TC-22	Verify preprocessed data is saved with operations	Complete a full diagnosis workflow	Data + operations stored in system	Fail
TC-23	Verify audit record created for doctor action	Complete a diagnosis session	Audit log entry with user, action, timestamp	Pass
TC-24	Navigate back to data query from report panel	Click back/return in report panel	Return to patient data query screen	Pass
TC-25	Admin accesses Admin Panel	Admin login, click 'Admin Panel'	Admin panel displayed	Pass
TC-26	Admin creates a new user	Fill in new user details, confirm	New user account created; audit record stored	Pass
TC-27	Admin modifies existing user settings	Select user, change settings, approve	Settings updated; audit record stored	Pass
TC-28	Non-admin user tries to access Admin Panel	Doctor account attempts admin access	Access denied; panel not shown	Pass
TC-29	Admin creates duplicate username	Enter existing username in new user form	Error message; user creation blocked	Pass
TC-30	Audit record captures admin identity on user creation	Admin creates a new user	Audit log shows admin name, action, timestamp	Pass

8.4 Test evaluation

Statistics (to be filled in by the team): Total number of defined tests N = 30(or updated by removing unused rows). Of these, F tests failed or B tests could not be run due to environment issues. The success rate can be reported based only on the tests that were run, calculated as $(N - F - B) / (N - B) \times 100$.

Observed errors and classification: If there are failed tests, they should be grouped as critical (input or security violation), medium (specific API or single environment), or low (interface text, edge case). Example: If GCS configuration is missing, media tests may appear to fail; this is more a result of missing environment configuration than a “product defect.”

Identified or known limitations: Partial feeding of patient/examination data with sample data on the client side may result in limited overlap between test results and the “end-to-end live system.” The partial integration of AI analysis may limit the full clinical validation of TC-18/TC-19 results.

Future improvements: Server-side controller and service tests using JUnit 5 and Spring Boot Test; PostgreSQL integration tests using Testcontainers; client-side end-to-end scenarios using Playwright or Cypress; a separate staging environment for media tests and a GCS emulator or test bucket; adding OWASP ZAP or similar tools to the pipeline for load and security scans is recommended.

9.Conclusions and Recommendations

In this study, the NeoAI platform was developed to support an end-to-end workflow that clinicians can monitor in a digital environment during multi-region ultrasound examinations. Within the scope of the project, the following were targeted on the server side: a REST API using Java and Spring Boot; stateless authentication using Spring Security and JWT; a relational and extensible data model using Spring Data JPA and PostgreSQL; and secure, time-controlled access to media objects using Google Cloud Storage. On the client side, React, Vite, and Material UI were used to provide a multi-step user flow, profile and management screens, and reporting components.

The primary deliverables achieved at the end of the project include: secure login and authorization, user and administrator operations, API endpoints related to examinations and media, and the technical infrastructure to initiate and query an analysis session. The architecture is suitable for future integration with a real AI inference service, the hospital's core data system, and a full-scale production cloud configuration.

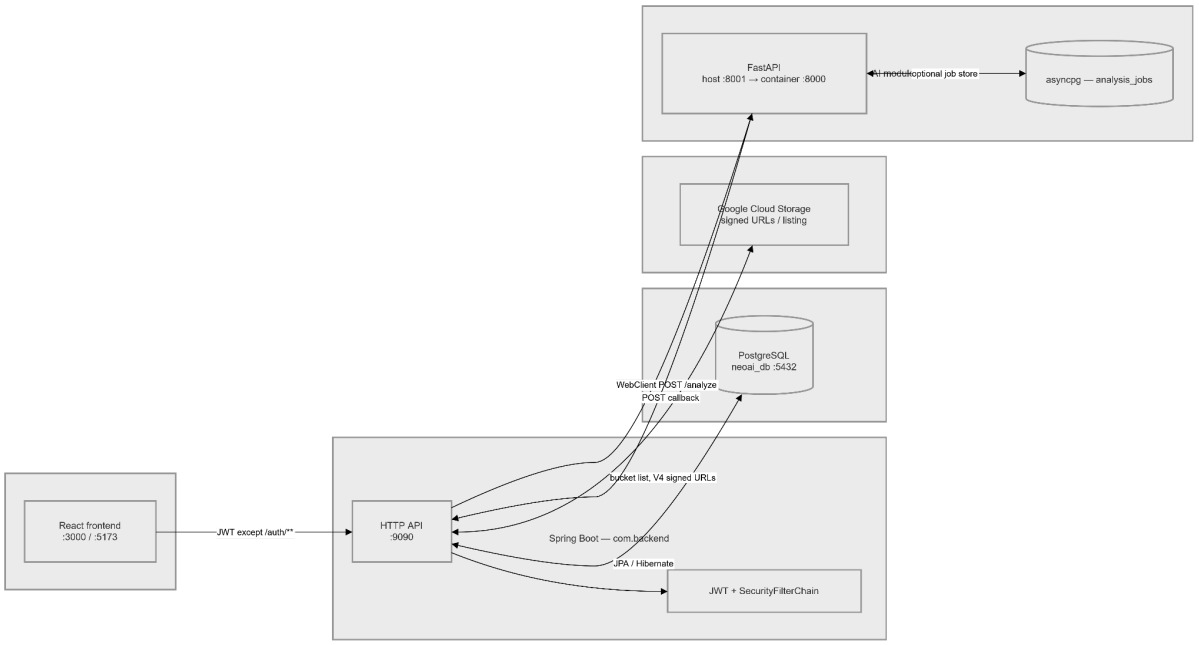
However, certain limitations must be clearly stated. During the development process, some patient and examination data on the client side was populated with sample data; as live integration is completed, these layers must be connected to the actual APIs. Clinical claims must be carefully limited until the AI module integration is fully completed at the production level. Until test automation is strengthened, the risk of regression remains dependent on manual testing.

Recommended future work includes: patient/visit integration with hospital systems via a standard interface (e.g., FHIR or an in-house API); clarification of the model service contract for analysis and observability (logs, metrics, monitoring) through asynchronous processing; adding unit and integration tests, as well as end-to-end tests, to the continuous integration pipeline; a GDPR-compliant data processing inventory and production security (HTTPS, key management, security headers).

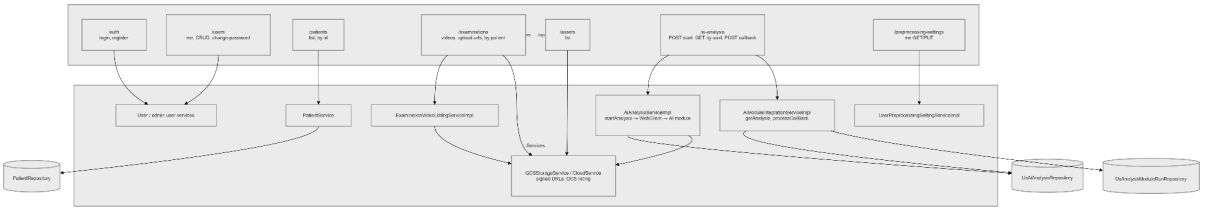
In conclusion, NeoAI is an engineering project executed through teamwork that concretizes ultrasound-focused clinical workflows using modern web technologies. This report aims to document the project's scientific and technical essence by bringing together the architecture, impacts, current issues, tools, research context, and testing perspective

10.Appendices

System Context(deployments & integrations)

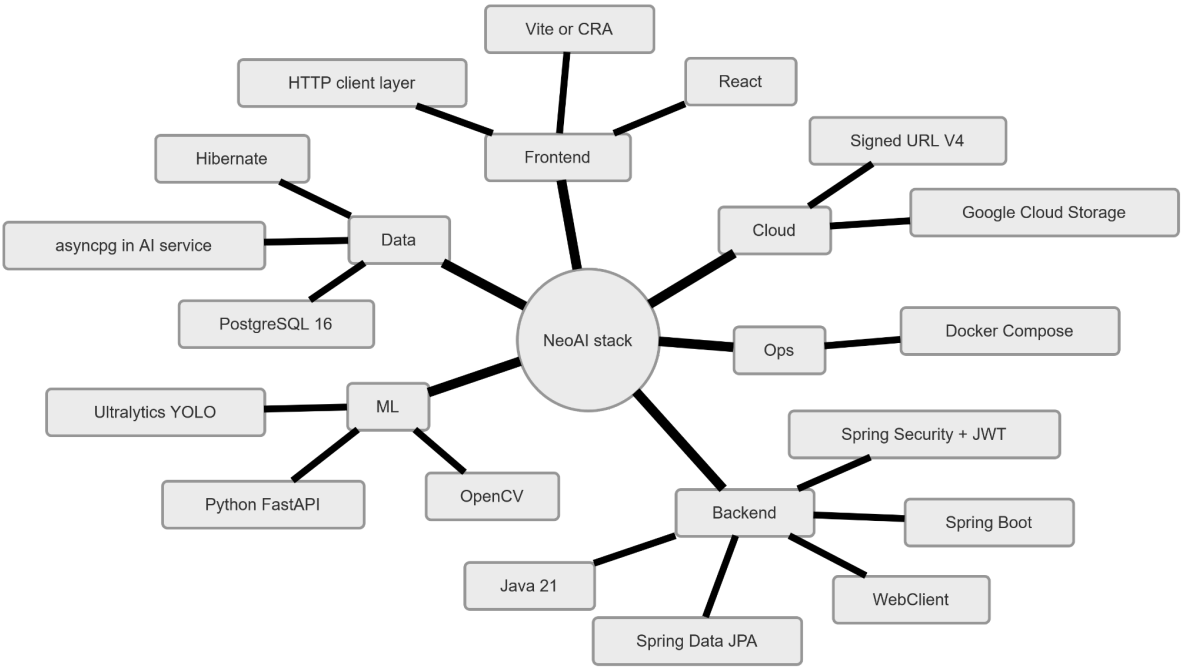


API surface -> Controllers -> services

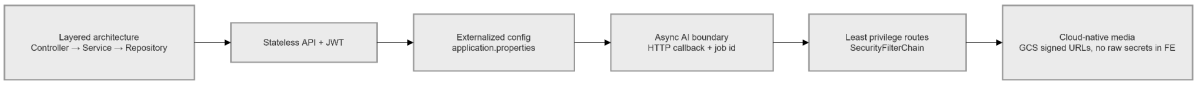


Async AI pipeline

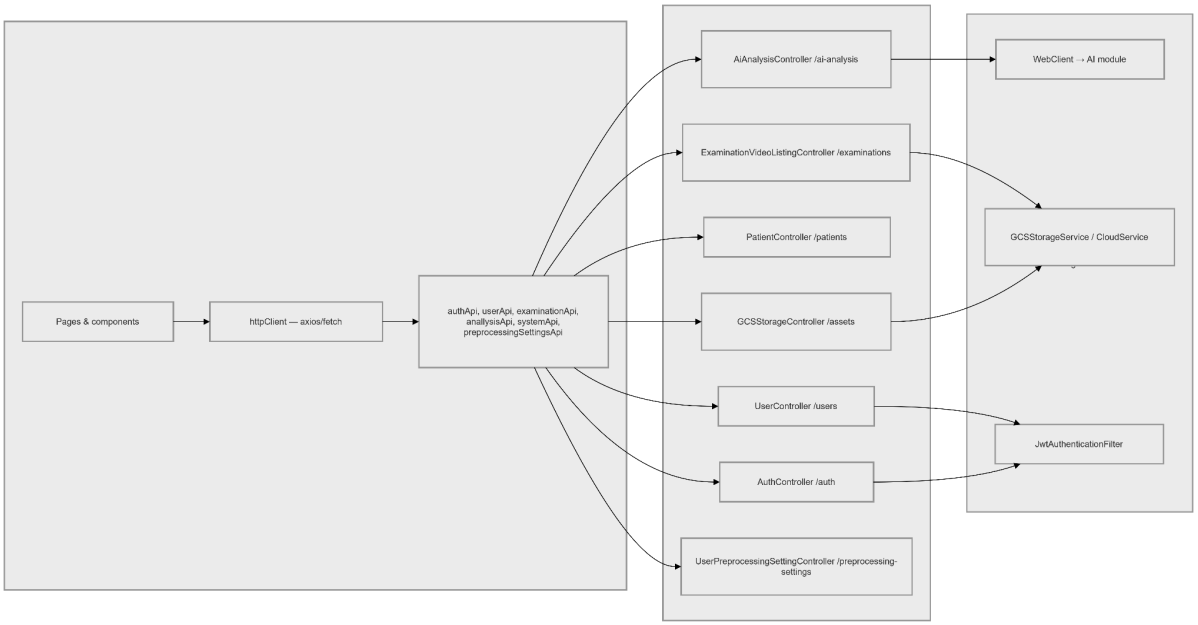
Technology Stack



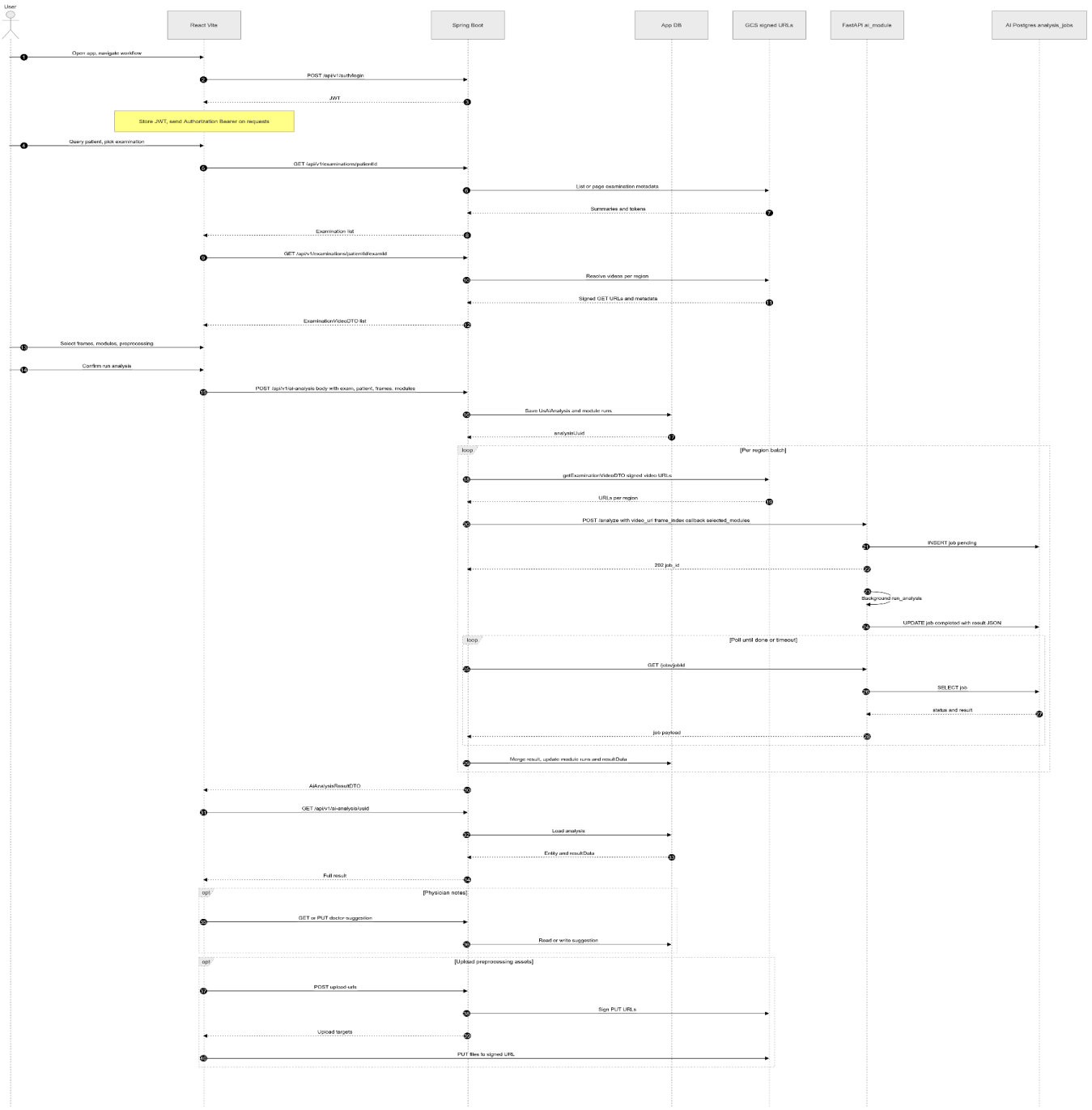
Engineering principles (design rationale)



Frontend Component Diagram



Sequence Diagram


User
React Vite
Spring Boot
App DB
GCS signed URLs
FastAPI ai_module
AI Postgres analysis_jobs
Open app, navigate workflow
POST /api/v1/auth/login
JWT
Store JWT, send Authorization Bearer on requests
Query patient, pick examination
GET /api/v1/examinations/patientId
List or page examination metadata
Summaries and tokens
Examination list
GET /api/v1/examinations/patientId/examId
Resolve videos per region
Signed GET URLs and metadata
ExaminationVideoDTO list
Select frames, modules, preprocessing
Confirm run analysis
POST /api/v1/ai-analysis body with exam, patient, frames, modules
Save UsAiAnalysis and module runs
analysisUuid
loop
[Per region batch]
getExaminationVideoDTO signed video URLs
URLs per region
POST /analyze with video_url frame_index callback selected_modules
INSERT job pending
202 job_id
Background run_analysis
UPDATE job completed with result JSON
loop
[Poll until done or timeout]
GET /jobs/jobId
SELECT job
status and result
job payload
Merge result, update module runs and resultData
AiAnalysisResultDTO
GET /api/v1/ai-analysis/uuid
Load analysis
Entity and resultData
Full result
opt
[Physician notes]
GET or PUT doctor-suggestion
Read or write suggestion
opt
[Upload preprocessing assets]
POST upload-urls
Sign PUT URLs
Upload targets
PUT files to signed URL

11.Reference

- European Parliament & Council of the European Union. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union. https://eur-lex.europa.eu/eli/reg/2024/1689/oj
- European Parliament & Council of the European Union. (2017). Regulation (EU) 2017/745 on medical devices. Official Journal of the European Union. https://eur-lex.europa.eu/eli/reg/2017/745/oj
- Google LLC. (t.y.). Cloud Storage: Authenticate with signed URLs. Google Cloud Documentation. Erişim: 17 Nisan 2026, https://cloud.google.com/storage/docs/authentication
- Google LLC. (t.y.). Cloud Storage: Signed URLs (V4). Google Cloud Documentation. Erişim: 17 Nisan 2026, https://cloud.google.com/storage/docs/access-control/signed-urls
- Meta Platforms, Inc. (t.y.). React — Learn React. React Documentation. Erişim: 17 Nisan 2026, https://react.dev/
- Mozilla Foundation. (t.y.). MDN Web Docs — Fetch API. MDN Web Docs. Erişim: 17 Nisan 2026, https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- OWASP Foundation. (t.y.). OWASP Top 10. OWASP. Erişim: 17 Nisan 2026, https://owasp.org/www-project-top-ten/
- PostgreSQL Global Development Group. (t.y.). PostgreSQL Documentation (Current). PostgreSQL. Erişim: 17 Nisan 2026, https://www.postgresql.org/docs/
- Republic of Turkey. (2016). Kişisel Verilerin Korunması Kanunu (6698 sayılı Kanun). Resmî Gazete. https://www.mevzuat.gov.tr/mevzuat?MevzuatNo=6698&MevzuatTur=1&MevzuatTertip=5
- Spring Authors. (t.y.). Spring Boot Reference Documentation. Spring. Erişim: 17 Nisan 2026, https://docs.spring.io/spring-boot/reference/
- Spring Authors. (t.y.). Spring Security Reference. Spring. Erişim: 17 Nisan 2026, https://docs.spring.io/spring-security/reference/
- The Internet Society. (2015). JSON Web Token (JWT). RFC 7519. https://www.rfc-editor.org/rfc/rfc7519
- Vite Team. (t.y.). Vite — Guide. Vite Documentation. Erişim: 17 Nisan 2026, https://vite.dev/guide/